Nmap Fundamentals & Enumeration

Study Notes — Host Discovery, Scan Types, OS/Service Detection, NSE Basics, Decoy Scans

1. Getting Started — Basic Connectivity Checks

Loopback Test

```
ping 127.0.0.1
```

Run in the Kali Linux terminal. Pinging the loopback address tests whether the TCP/IP stack is functioning correctly on the local machine.

Testing Reachability to Windows

```
ping <windows-ip>
```

If the response is good, connectivity works. Otherwise, the Windows Firewall might be blocking ICMP packets coming from the Kali VM.

Checking Kali's Own Network Configuration

```
ip addr
ip route
```

Why `ip addr`?

It tells us:

- Does Kali even have an IP address?
- Which interface is being used (eth0, wlan0, etc.)?
- What subnet is it on?

Example output:

```
eth0
inet 10.35.246.241/24
```

Terminal output of `ip addr` showing Kali's assigned IP on eth0.

Immediately learned:

- Kali successfully got an IP.

- The interface is up.
- Kali is on the 10.35.246.0/24 network.

Why `ip route`?

`ip route` tells us where Kali sends packets. Example output:

```
default via 10.35.246.154
10.35.246.0/24 dev eth0
```

1. Local network route

```
10.35.246.0/24 dev eth0
```

Means: "Anything with an IP starting with 10.35.246.x is on my local network." Since Windows was `10.35.246.91`, Kali knows Windows is local and should send packets directly.

2. Default gateway

```
default via 10.35.246.154
```

Means: "If the destination isn't local, send it to the router." So internet traffic has a valid path.

Investigating a Failed Ping to Windows

Before proceeding, I checked if the Windows host could reach the VM by pinging it — in our case it could. The most likely explanation is that **Windows Firewall was blocking incoming ICMP Echo Requests (ping) while still allowing outgoing pings**. This is actually very common on Windows.

To confirm Kali's networking itself was fine, I pinged the default gateway (`10.35.246.154`, found via `ip route`) from Kali. It worked, which told us:

- Kali can reach the gateway (10.35.246.154).
- Kali has a valid IP (10.35.246.241).
- The network adapter is working.
- Routing is working.

So the problem was not Kali's networking — it was Windows blocking ping requests.

2. Host Discovery Basics

Pinging via Nmap Instead

```
nmap -sP 10.35.246.91  (HOST IP)
```

Result:

```
Host is up (0.00056s latency).
MAC Address: 50:5A:65:4F:14:E9 (AzureWave Technology)
Nmap done: 1 IP address (1 host up) scanned in 0.09 seconds
```

> **Note:** `-sP` (now replaced by `-sn` in modern Nmap) performs host discovery only. It answers one question — "Which hosts are alive?" — and does not perform a port scan.

The -Pn Flag

`-Pn` does exactly one thing: it disables host discovery.

Normally Nmap does:

```
Is the host alive?
      ↓
     Yes
      ↓
  Scan ports
```

With `-Pn`, Nmap does:

```
Skip "Is the host alive?"
```

```
    ↓
Assume it is alive
    ↓
Scan ports
```

```
nmap -Pn 10.35.246.91
```

Why the Host Showed as "Up" Despite the Ping Being Blocked

Running a normal scan:

```
nmap 10.35.246.91
```

gave:

```
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-28 22:39 IST
Nmap scan report for 10.35.246.91
Host is up (0.00036s latency).
All 1000 scanned ports on 10.35.246.91 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 50:5A:65:4F:14:E9 (AzureWave Technology)
Nmap done: 1 IP address (1 host up) scanned in 21.36 seconds
```

This meant it detected the host, so the earlier ICMP failure wasn't a host-discovery issue — it was a firewall just filtering and dropping the packet.

How did Kali know the host was up, then?

Because Nmap didn't need ICMP to discover the Windows machine. Both machines were on the same local network:

- Windows: 10.35.246.91
- Kali: 10.35.246.241
- Subnet: 10.35.246.0/24

On a local network, Nmap usually uses **ARP** for host discovery:

```
Kali:  "Who has 10.35.246.91?"   (ARP Request — broadcast)
Windows: "I am 10.35.246.91, my MAC is 50:5A:65:4F:14:E9"  (ARP Reply)
```

Once Nmap gets that ARP Reply, it knows the device exists, is powered on, and is on the local LAN — so it reports "Host is up."

> **Note:** Whether "the firewall allowed it" or "the firewall doesn't filter ARP" depends on implementation details. For practical purposes, the important thing is: ARP traffic was not blocked, but ICMP ping traffic was blocked.

3. TCP Connect Scan (-sT)

Example Command

```
sudo nmap -sT -p 80,443 10.35.246.0/24
```

- `sudo` → Run as administrator.
- `nmap` → Start Nmap.
- `-sT` → Perform a TCP Connect Scan.
- `-p 80,443` → Scan only ports 80 (HTTP) and 443 (HTTPS).
- `10.35.246.0/24` → Scan every IP from 10.35.246.1 to 10.35.246.254.

Step 1: Host Discovery

Before scanning ports, Nmap first asks: "Which hosts are alive?" Since this was a local LAN scan, it used ARP and discovered 3 live hosts: `10.35.246.91`, `10.35.246.154`, `10.35.246.242`.

Step 2: Port Scan Results per Host

Host 1 — Windows (10.35.246.91)

```
PORT    STATE     SERVICE
80/tcp  filtered  http
443/tcp filtered  https
```

Nmap sent a SYN to port 80. Expected responses: SYN/ACK (open) or RST (closed). Instead: no response, so Nmap concludes **filtered** — something (usually a firewall) prevented Nmap from determining whether the port is open or closed.

Host 2 — Router (10.35.246.154)

```
PORT    STATE
80/tcp  closed
443/tcp closed
```

Nmap sent a SYN; the router replied with RST, meaning "I received your packet, but nothing is listening on this port," so Nmap marks it **closed**.

Host 3 — Kali VM (10.35.246.242)

Exactly the same — 80 and 443 both closed. The Kali machine answered the scan and said "No web server is running."

Why Windows Showed "Filtered"

Because the Windows firewall is likely configured like this:

```
Incoming SYN
     ↓
Windows Firewall
     ↓
     Drop
     ↓
  No response
```

Nmap cannot distinguish between a firewall dropping packets, a firewall silently ignoring packets, or some other network device filtering packets — so it simply reports **filtered**.

```
Kali
 |
 ├── Discover hosts (ARP)
 |     ├── Windows ✓
 |     ├── Router  ✓
 |     └── Kali    ✓
 |
 └── Scan ports 80 and 443
       ├── Windows
       |     SYN → Firewall → Drop
       |     → filtered
       |
       ├── Router
       |     SYN → RST
       |     → closed
       |
       └── Kali
             SYN → RST
             → closed
```

4. TCP Connect Scan vs SYN Scan

The Problem with -sT

A TCP Connect scan might be more easily detected by an IDS or firewall because many services log successful TCP connections. Example:

```
Web Server Log
Client connected from: 10.35.246.5
```

A Connect Scan often creates an actual connection that applications may log.

SYN Scan (-sS)

```
sudo nmap -sS -p 80,443 10.35.246.0/24
```

In a SYN scan, the last ACK flag is never sent — instead Nmap sends RST. A SYN Scan usually doesn't establish the connection, so many application-level logs won't record it (though firewalls, IDS/IPS, or packet capture systems still can). This is why SYN scans

are often called **"half-open scans."**

> **Important:** They are not invisible. Modern security devices frequently detect SYN scans.

Why Use SYN Scan?

Reason 1 — Faster: A SYN scan skips the final ACK and avoids creating a full TCP session. When scanning thousands of ports across hundreds of machines, this saves time and system resources.

Reason 2 — Less logging: As above — many application-level logs won't record it.

Reason 3 — More control: A SYN scan lets Nmap craft and analyze packets directly, making it easier to detect filtered ports, interpret unusual responses, and perform more advanced scanning techniques. With `-sT`, the operating system handles much of the connection process, so Nmap has less direct control.

When Would You Use -sT Instead?

Sometimes a SYN scan isn't possible:

- **You are not root** — On Linux, `nmap -sS` requires raw packet privileges (typically root). As a normal user, `nmap -sT` still works because it relies on the OS's standard networking APIs.
- **Raw sockets are restricted** — Some environments restrict raw packet creation, making `-sT` the only option.

Example of scale benefit: Scanning 100 hosts × 1000 ports each = 100,000 port checks. A SYN scan avoids completing tens of thousands of TCP connections, making it far more efficient.

Which Is Preferred for Ethical Hacking?

- **-sS (SYN Scan)** → Preferred when you have the necessary privileges. Faster, uses fewer resources, gives Nmap greater control.
- **-sT (TCP Connect Scan)** → Used when raw packet scanning isn't available, e.g. insufficient privileges.

> **Note:** Sometimes not all hosts show up consistently between scans. In my case, the first scan found only 2 hosts "up," but the second scan found 3.

Why Host Counts Can Differ Between Scans

Nmap's process is: Host discovery ("Is this IP alive?") → Port scan (only if yes).

First scan:

```
10.35.246.154
      ↓
No ARP reply (or reply arrived too late)
      ↓
Nmap assumed it was down
      ↓
No port scan performed
```

Second scan:

```
10.35.246.154
      ↓
ARP reply received
      ↓
Nmap marked it as "Host is up"
      ↓
Scanned ports 80 and 443
```

So the difference wasn't `-sS` vs `-sT` — the device simply didn't respond to the host discovery phase the first time but did the second time. This can happen because the device was sleeping, temporarily busy, or due to a small timing variation on the network.

5. Wireshark Troubleshooting Lesson

While learning Nmap and Wireshark, I performed both TCP Connect (`-sT`) and SYN (`-sS`) scans from my Kali VM to my Windows host.

Initially, all scanned ports appeared filtered. I captured traffic on the Windows host using Wireshark (Wi-Fi interface), but I only

saw reverse DNS (PTR) queries from Nmap and no TCP SYN packets, which was unexpected.

To investigate, I:

- Verified that both machines were on the same subnet.
- Confirmed the VM was using Bridged Adapter mode.
- Used `tcpdump` on Kali and confirmed that the SYN packets were actually leaving the VM.
- Temporarily disabled the Windows Defender Firewall and reran the scan.

After disabling the firewall, `nmap -sT` successfully identified several open ports, proving that full TCP handshakes had occurred. I also successfully pinged the Windows host, confirming ICMP traffic was working normally.

Despite successful TCP connections and ICMP communication, Wireshark on the Windows host still failed to capture the TCP and ICMP packets.

Final Conclusion

The issue was not with Nmap, Kali, the network configuration, or the scan commands. The most likely cause is that the Windows packet capture path (Npcap/Wi-Fi adapter/driver combination) did not expose the traffic to Wireshark. Since `tcpdump` on Kali captured the SYN packets and Nmap successfully completed TCP connections, the network communication itself was functioning correctly.

> **Lesson learned:** If expected packets are not visible in Wireshark, it does not necessarily mean they were never transmitted. Packet capture tools depend on the operating system, capture driver, and network adapter. It is important to verify observations using multiple tools (e.g., tcpdump, Nmap results, ping) before concluding that packets were not sent or received.

6. Nmap Enumeration Lab Summary (Kali → Windows Host)

Objective: Perform multiple Nmap scans from Kali against a Windows host to understand how firewall settings affect scan results and how different Nmap options reveal increasing amounts of information.

Step 1 — Initial TCP Port Scan (Firewall Enabled)

Some ports appeared as filtered — e.g. port 80 → filtered, port 443 → filtered.

What I learned: Filtered means the probe reached the target, but a firewall or filtering device prevented Nmap from determining whether the port is open or closed. The Windows Firewall was silently dropping or filtering the packets.

Step 2 — Scan After Disabling Windows Firewall

```
Not shown: 993 closed tcp ports (reset)
Open ports:
135
139
445
902
912
1521
8080
```

What I learned: Unlike the firewall-enabled scan, Windows now responded to closed ports with TCP RST (Reset) packets, telling Nmap "the host is alive, but nothing is listening on this port." As a result, closed ports were identified correctly, less information was hidden, and enumeration became easier.

Step 3 — OS Detection (-O)

```
sudo nmap -O <target-ip>
```

Result: `No exact OS matches for host`

What I learned: This does not mean the host is not Windows — it simply means Nmap's TCP/IP fingerprint did not exactly match an entry in its OS fingerprint database. Possible reasons: different Windows build, updated network stack, VMware/network drivers, or a fingerprint that isn't distinctive enough.

Step 4 — Advanced Scan

```
sudo nmap -sV -O -A <target-ip>
```

This enabled: Service Version Detection (`-sV`), OS Detection (`-O`), Default NSE Scripts, and Traceroute.

Step 5 — Service Detection Results

Port	Service	Notes
135	Microsoft Windows RPC	Windows Remote Procedure Call service.
139	Microsoft Windows netbios-ssn	NetBIOS Session Service — used for legacy Windows networking.
445	Microsoft-DS (SMB)	Used for file sharing, printer sharing, Windows networking.
902	VMware Authentication Daemon	VMware authentication service.
912	VMware Authentication Daemon	Another VMware auth service — indicates VMware is installed on the Windows machine.
1521	Oracle TNS Listener 11.2.0.2.0	Listener responded; authentication required (`unauthorized`) — Nmap could not continue without credentials.
8080	Oracle XML DB Enterprise Edition httpd	Returned `HTTP/1.1 401 Unauthorized`; discovered `Basic realm = XDB` (HTTP Basic Auth) and server header Oracle XML DB / Oracle Database.

Step 6 — Hostname Discovery

NSE returned: `NetBIOS name: HIMAL` — the Windows computer name was discovered without logging into the machine.

Step 7 — SMB Security Check

Nmap reported: `Message signing enabled but not required`. Meaning: SMB signing is supported but optional — mandatory signing provides stronger protection against certain attacks.

Step 8 — SMB Time

Nmap retrieved the system time from the SMB service — useful for checking clock synchronization, authentication troubleshooting, and certain penetration testing techniques.

Step 9 — Traceroute

Result: `Network Distance: 1 hop` — meaning the Kali VM and Windows host are on the same local network.

Step 10 — MAC Address

Nmap showed AzureWave Technology — this identifies the manufacturer of the network adapter, not the operating system.

Step 11 — Windows Firewall Comparison

Firewall Enabled	Firewall Disabled
Some ports appeared filtered.	Closed ports replied with TCP RST packets.
Firewall silently blocked packets.	Open ports were identified more accurately.
Less information was revealed.	Service detection worked better.
Enumeration became more difficult.	Much more information became available.

Key Concepts Learned

- **Open Port:** A service is actively listening.
- **Closed Port:** No service is listening, but the host responds with a TCP Reset.
- **Filtered Port:** A firewall or filtering device prevents Nmap from determining the port's state.
- **Service Detection (-sV):** Identifies the application and version running on open ports.
- **OS Detection (-O):** Uses TCP/IP fingerprinting to estimate the operating system.
- **Aggressive Scan (-A):** Combines service detection, OS detection, default NSE scripts, and traceroute.
- **401 Unauthorized:** The server is reachable but requires authentication.
- **Basic Authentication:** The server expects a username and password using the HTTP Basic Authentication mechanism.

Enumeration Workflow Learned

```
1. Host Discovery      (-sn)
```

```
2. Port Scan           (-sS or -sT)
3. Service Detection    (-sV)
4. OS Detection         (-O)
5. Default NSE Scripts  (-sC or -A)
6. Targeted enumeration based on discovered services
```

This lab demonstrated how each additional scan option reveals progressively more information about a target host.

7. OS Detection (-O) — Deep Dive

```
sudo nmap -O 10.35.246.68
```

Purpose: Nmap does not ask the target what OS it is running. Instead, it infers the OS by analyzing the target's TCP/IP stack behavior and comparing it against Nmap's OS fingerprint database.

What Happens Internally

```
Parse command
      ↓
Determine route/interface
      ↓
Resolve MAC address (ARP if needed)
      ↓
Host discovery
      ↓
TCP port scan
      ↓
Find open and closed ports
      ↓
Send OS fingerprint probes
      ↓
Analyze TCP/IP responses
      ↓
Build fingerprint
      ↓
Compare with Nmap's OS fingerprint database
      ↓
Display the closest OS match
```

Why Nmap Needs Both Open and Closed Ports

Nmap does not care about the ports themselves — it cares about how the operating system responds when packets are sent to an open port vs. a closed port. Each type reveals different characteristics of the TCP/IP stack, giving Nmap more information for a more accurate fingerprint.

Combination	Accuracy
Open port + Closed port	✅ Best accuracy
Only open ports	✅ Lower accuracy
Only closed ports	✅ Lower accuracy
Only filtered ports	❌ Poor accuracy or detection may fail

Filtered ports provide less information because firewalls often drop packets instead of allowing the TCP/IP stack to respond.

What Nmap Analyzes During OS Detection

- **TTL (Time To Live)** → Helpful for identifying the OS and estimating hop count.
- **TCP Window Size** → Another fingerprint characteristic.
- **TCP Options** → One of the strongest fingerprinting indicators.
- **TCP Sequence Number Generation** → Shows how the OS generates TCP sequence numbers.
- **IP ID Generation** → Another fingerprint characteristic.
- **RST Behavior** → Shows how the OS responds when a closed port is contacted.
- **ECN Support** → Minor fingerprinting characteristic.

Nmap combines all of these characteristics into a single fingerprint.

Key Takeaways

- Host discovery and OS detection are different processes.
- Host discovery on a local LAN mainly uses an ARP reply.
- OS detection relies on TCP/IP fingerprinting.
- Nmap identifies an OS by comparing the target's network behavior with its fingerprint database.
- Best accuracy comes from analyzing responses from both an open TCP port and a closed TCP port.
- The *responses* — not the ports themselves — are what Nmap uses to identify the operating system.

8. Host Discovery — ARP Mechanics

Purpose: Host discovery determines whether a target device is alive before performing further scans.

Host Discovery on a Local Network

When scanning a host on the same LAN, Nmap primarily uses ARP requests. Process:

1. Nmap sends an ARP request asking: "Who has this IP address?"
2. If the target owns that IP, it sends an ARP reply.
3. After receiving the reply, Nmap concludes the host is up, and learns the target's MAC address from the same reply.

> **Important concept:** The ARP reply is what proves the host is alive. The MAC address is information contained within the ARP reply, not the reason Nmap decides the host is up.

Hypothetical Thought Experiment

If an imaginary version of ARP allowed a reply without a MAC address (impossible in the real protocol), Nmap would still conceptually determine the host is up because it received a reply. The missing MAC address would only mean it could not identify the hardware address. This shows the key factor in host discovery is *receiving a response*, not the MAC address itself.

Remote Networks

When the target is on a different network, ARP cannot be used because routers do not forward ARP packets. Instead, Nmap uses other probes such as:

- ICMP Echo Requests (ping)
- TCP SYN probes
- TCP ACK probes

If the target responds to one of these, Nmap concludes the host is up, even though it cannot learn the target's MAC address.

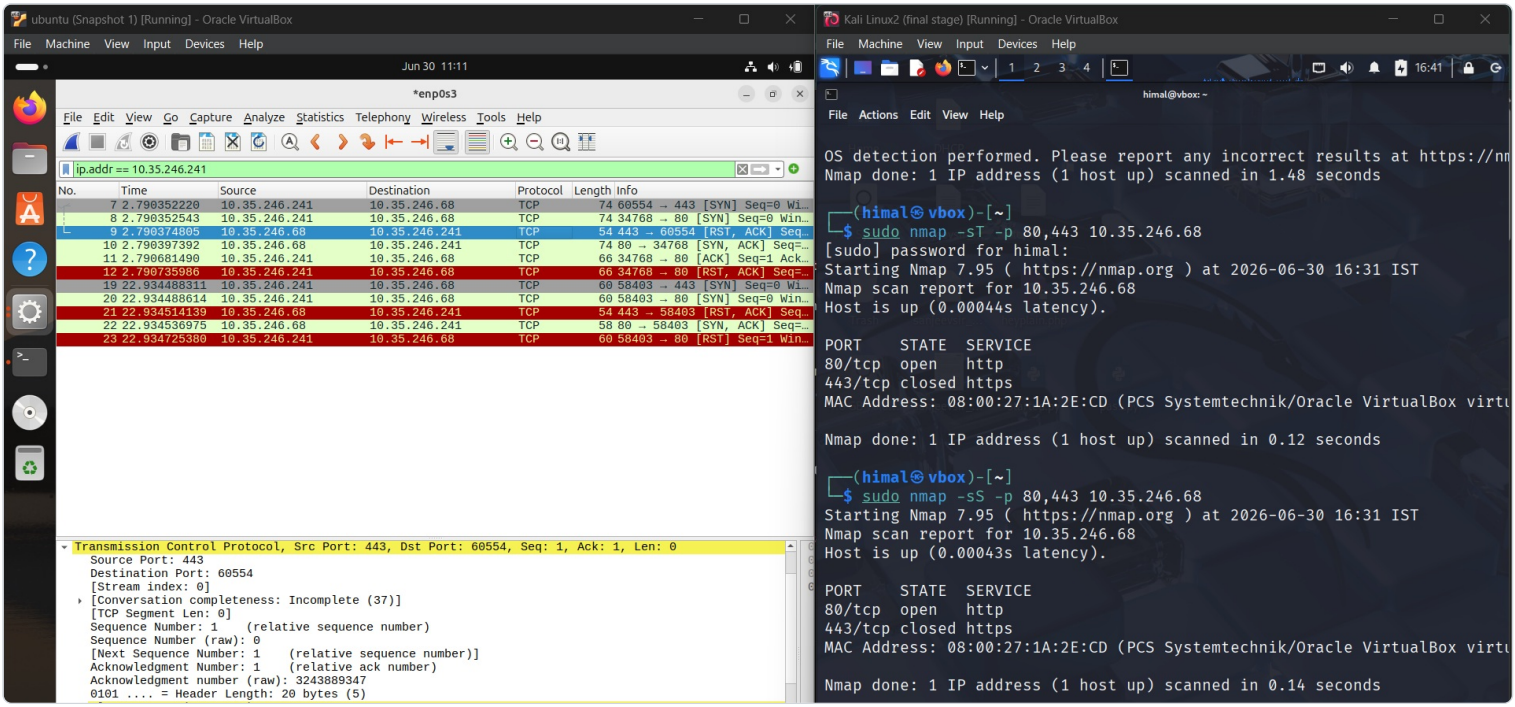
Key Takeaway

- **Same LAN:** Host discovery typically uses ARP. A valid ARP reply tells Nmap the host is up and provides the MAC address.
- **Different network:** Host discovery uses other protocols (ICMP or TCP) because ARP only works on the local network.

9. Wireshark Comparison: TCP Connect vs SYN Scan

This section is a representation of Wireshark traffic for the TCP connect and SYN scan, which wasn't captured on the Windows host but was successfully captured on the Ubuntu VM's Wireshark.

Setup: Two VMs — a Kali Linux attacker machine (10.35.246.241) and an Ubuntu target (10.35.246.68), both analyzed with Wireshark on the Ubuntu side.



Left: Wireshark capture on the Ubuntu VM showing the SYN/SYN-ACK/RST exchange. Right: the Kali terminal running the -sT and -sS scans against the Ubuntu target.

Scans Performed on Kali (against 10.35.246.68)

TCP Connect scan:

```
sudo nmap -sT -p 80,443 10.35.246.68
```

Completes the full TCP three-way handshake for each port. Result: port 80/tcp open (http), port 443/tcp closed (https).

TCP SYN scan (stealth scan):

```
sudo nmap -sS -p 80,443 10.35.246.68
```

Sends only a SYN packet, doesn't complete the handshake (half-open scan). Same result: 80 open, 443 closed.

What Wireshark Captured (filter: ip.addr == 10.35.246.241)

- **Frames 7–12 (first scan, -sT):** SYN to port 443 gets RST,ACK back (closed), while SYN to port 80 gets SYN,ACK followed by a full handshake (ACK) and then a RST,ACK to tear it down — consistent with `-sT` completing the connection before closing it.
- **Frames 19–23 (second scan, -sS):** Same pattern of SYN → RST,ACK for 443 (closed) and SYN → SYN,ACK for 80. Typically Nmap's SYN scan would just send a final RST instead of completing the handshake — worth double-checking whether the capture showed a full ACK exchange for port 80 in this set too, since that would suggest the tool/version behaved more like a connect scan, or there was an additional connection.

> **Key teaching point:** This pairing visually demonstrates the difference between `-sT` (full handshake, visible as SYN→SYN-ACK→ACK→RST) and `-sS` (half-open, normally just SYN→SYN-ACK→RST, no final ACK) against the same target/ports, letting you compare the packet-level footprint of each scan type.

10. Aggressive Scan (-A) — Ubuntu Lab

```
sudo nmap -A <ubuntu-ip>
```

What -A Enables

`-A` is a shortcut that enables multiple advanced features in one scan:

- `-O` → OS detection
- `-sV` → Service/version detection
- `-sC` → Runs the default NSE (Nmap Scripting Engine) scripts
- `--traceroute` → Performs traceroute to estimate network distance

It is called an **Aggressive Scan** because it sends more probes, performs deeper enumeration, takes longer, generates more network traffic, and is more likely to be detected than a basic scan. It is **not** an attack.

Understanding the Scan Output

Host is up — the target machine responded to Nmap's probes and is reachable.

Closed Ports: `996 closed tcp ports (reset)` — responded with RST packets, indicating nothing is listening.

Open Ports on the Ubuntu VM

Port	Service
22/tcp	SSH
80/tcp	HTTP (Apache)
139/tcp	NetBIOS/SMB
445/tcp	SMB

These become the main focus of further enumeration during a penetration test.

Service Version Detection

Nmap identified exact software versions, e.g. OpenSSH 9.6p1, Apache 2.4.58. Knowing the exact version allows research into known vulnerabilities (CVEs), misconfigurations, and publicly available exploits.

OS Detection

```
Running: Linux 4.X|5.X
OS details: Linux 4.15 - 5.19
```

Nmap estimates the OS by comparing target responses to specially crafted packets against its fingerprint database.

OS CPE

Example: `cpe:/o:linux:linux_kernel:5`

A CPE (Common Platform Enumeration) is a standardized identifier for software or operating systems. It helps security tools and vulnerability scanners match software with known CVEs.

Network Distance (Hop Count)

Example: `Network Distance: 1 hop`

A hop is each Layer 3 device the packet reaches. Nmap counts the destination host itself as the final hop:

- 1 hop = destination host only (no routers between me and the target)
- 2 hops = one router + destination
- 3 hops = two routers + destination

Why the Phone Hotspot Wasn't Counted as a Hop

Both VMs were configured in Bridged Adapter mode and received IP addresses on the same subnet (e.g. Kali: 10.35.246.91, Ubuntu: 10.35.246.68). Since both belong to the same subnet, Kali determines the destination is local. Instead of sending packets to the default gateway (the phone), Kali:

- Uses ARP to learn Ubuntu's MAC address.
- Sends the Ethernet frame directly to Ubuntu.

The phone only acts as the default gateway for traffic leaving the subnet (e.g. internet traffic). Therefore the traceroute showed only Hop 1 = Ubuntu VM — the phone was not a hop because it never routed the packets.

Same Subnet Rule

If the destination IP is on the same subnet:

- The default gateway is not used.
- ARP is used to obtain the destination MAC address.
- The frame is delivered directly.

If the destination IP is on a different subnet:

- The packet is sent to the default gateway.
- The gateway becomes the first hop.

SSH Host Keys

```
ssh-hostkey:
  ECDSA
  ED25519
```

These are the public host keys of the SSH server. Every SSH server has a host private key (secret; never leaves the server) and a host public key (shared with clients). Nmap only displays the public host key.

Purpose: SSH host keys authenticate the SSH server to the client — proving the client is communicating with the correct server. They do not authenticate the user. Authentication flow: the server proves its identity using its host key, then the client authenticates itself using a password or its own SSH key.

Can SSH host keys be used to gain SSH access? No. Knowing the server's public host key does not allow logging into the server, recovering the private key, or bypassing authentication. The private host key never leaves the server.

Ethical hacking use: SSH host keys are mainly useful for identification, not exploitation. They help:

- Confirm you're communicating with the same SSH server across multiple scans.
- Detect if a server has been rebuilt or replaced.
- Document the target during a penetration test.
- Recognize the same server even if its IP address changes.

They are not normally used to exploit the target.

Which device owns the SSH host key? The SSH host key always belongs to the SSH server. In this lab: Kali VM (client) → Ubuntu VM (SSH server). So the SSH host keys displayed by Nmap belong to the Ubuntu VM, because Ubuntu is running the SSH service. If the host key changes unexpectedly, it may indicate the server was rebuilt, SSH was reinstalled, host keys were regenerated, a different machine now owns that IP, or (in rare cases) a possible man-in-the-middle attack.

11. Nmap Decoy Scan (-D)

Command Used

```
sudo nmap -sS -p 80,443 -D 10.35.246.111 10.35.246.68
```

- `-sS` → TCP SYN (half-open) scan.
- `-p 80,443` → Scan only ports 80 and 443.
- `-D 10.35.246.111` → Use 10.35.246.111 as a decoy IP.
- `10.35.246.68` → Target (Ubuntu).

What a Decoy Scan Does

A decoy scan makes the target believe multiple IP addresses are scanning it. Nmap sends probe packets that appear to originate from both the real IP address and the decoy IP address. The purpose is to make it harder for someone looking only at the target's logs to immediately identify which IP is the real scanner.

What Was Observed in Wireshark

SYN packets arrived at the Ubuntu target from both the real Kali IP and the decoy IP (10.35.246.111), confirming the decoy packets were actually transmitted on the network.

Port 80 (Open):

```
Kali → SYN
Ubuntu → SYN-ACK
Kali → RST
```

Since `-sS` is a half-open scan, Nmap did not complete the handshake — after receiving SYN-ACK it immediately sent RST.

Port 443 (Closed):

```
Kali → SYN
Ubuntu → RST-ACK
```

Receiving a RST-ACK indicated port 443 was closed.

Why Ubuntu Did Not Reply to the Decoy

The decoy IP was on the same local subnet. Before Ubuntu could send a response to the decoy IP, it needed the decoy's MAC address, so it sent ARP requests asking "Who has 10.35.246.111?" Since no device actually owned that IP address, no ARP reply was received. Without knowing the destination MAC address, Ubuntu could not send the response packet to the decoy.

Therefore: the SYN packets from the decoy reached Ubuntu; Ubuntu attempted to reply; but it could not send the reply because ARP resolution failed. In Wireshark, ARP requests for the decoy IP were visible but no response packets from Ubuntu to the decoy.

Important Conclusion

The decoy successfully generated incoming scan traffic toward the target. However, because the decoy IP did not belong to a real host on the LAN, Ubuntu could not send responses back to it. This reduces the effectiveness of the decoy on a local network, because someone inspecting the packet capture may notice that the decoy IP never answers ARP requests.

Overall Understanding

Verified through Wireshark that Nmap's decoy scan really sends packets that appear to originate from both the real IP and the decoy IP. Also learned that on a local subnet, the target must perform ARP resolution before replying. If the decoy IP does not exist, ARP fails, preventing the target from sending responses to the decoy. Although the decoy still creates additional apparent scan sources, an investigator examining local network traffic could notice that the decoy IP never responded to ARP — making it less convincing than a decoy that belongs to a real, active host.

12. -sC, NSE, and Saving Scan Results

-sC (Default Scripts)

`-sC` tells Nmap to run its default set of NSE (Nmap Scripting Engine) scripts after discovering open ports. These scripts automatically gather additional information about the services running on the target. Instead of only reporting that a port is open, they can provide details such as webpage titles, SSL information, SMB details, and more.

`-sC` is commonly combined with `-sV`:

```
nmap -sV -sC <target>
```

NSE (Nmap Scripting Engine)

NSE is a scripting framework built into Nmap. It allows Nmap to perform tasks beyond simple port scanning by interacting with discovered services. Scripts can:

- Enumerate services
- Gather additional information
- Check for known vulnerabilities
- Retrieve configuration details
- Perform many other service-specific tasks

NSE Scripts

`ftp-anon` is an individual NSE script, not a category. Many scripts are designed for specific services:

Category	Example Scripts
FTP	ftp-anon, ftp-syst, ftp-bounce
HTTP	http-title, http-headers, http-enum
SMB	smb-os-discovery, smb-enum-shares, smb-enum-users

The naming usually indicates which service the script is intended to work with:

```
nmap --script ftp-anon <target>
```

This runs only the `ftp-anon` script.

Saving Scan Results

Nmap allows scan results to be saved while the scan is running. The output option is specified **before** starting the scan, not after:

```
nmap -sS -sV -oN results.txt <target>
```

When the scan begins, Nmap performs the scan, displays results in the terminal, and simultaneously writes the same results to `results.txt`. No separate save command is required after the scan finishes.

Output Options

Format	Command
Normal output	`nmap -oN results.txt <target>`
XML output	`nmap -oX results.xml <target>`
Grepable output	`nmap -oG results.gnmap <target>`
All major formats	`nmap -oA results <target>`

`-oA` creates: `results.nmap`, `results.xml`, `results.gnmap`

Key Points

- `-sC` runs Nmap's default NSE scripts.
- NSE is the scripting engine that extends Nmap's capabilities beyond port scanning.
- `ftp-anon` is a single NSE script, not a script category.
- Many NSE scripts are service-specific (FTP, HTTP, SMB, SSH, DNS, etc.).
- Output files are specified before the scan starts.
- Nmap saves the results automatically as the scan runs.
- Each scan only saves its own output — future scans must include an output option again if they also need to be saved.

13. Service Detection (-sV) vs OS Detection (-O)

Service Detection (-sV)

Purpose: Identify the service running on an open port and, if possible, its version.

How it works internally:

1. Nmap first discovers an open port.
2. It sends protocol/application-layer probes to that port.
3. The service responds with banners, headers, error messages, or other protocol-specific replies.
4. Nmap analyzes these responses.
5. It compares them against its service fingerprint database.
6. Nmap reports the most likely service and version.

Examples of information analyzed: service banners, HTTP headers, protocol responses, error messages, general application behavior.

> **Summary:** Service detection works by sending application-layer probes, analyzing the service's responses, and comparing them with Nmap's service fingerprint database to identify the service and version.

OS Detection (-O)

Purpose: Identify the operating system running on the target.

How it works internally: Nmap sends specially crafted TCP/IP probes and analyzes how the OS's network stack responds, examining characteristics such as:

- TCP options
- TCP window size
- Window scale
- Initial TTL
- DF (Don't Fragment) behavior
- TCP sequence number generation
- ICMP responses
- Other TCP/IP implementation details

Nmap compares these characteristics against its OS fingerprint database and reports the closest matching OS.

> **Summary:** OS detection works by analyzing the behavior of the target's TCP/IP stack and comparing those characteristics with

> Nmap's OS fingerprint database.

Key Difference

-sV	-O
Focuses on the application/service.	Focuses on the operating system.
Uses application-layer probes.	Uses specially crafted TCP/IP probes.
Fingerprints the service.	Fingerprints the OS's network stack.

Why -sV Is Commonly Combined with NSE Scripts

The primary reason is efficiency — you can gather service/version information and script results in a single scan. Additionally, some NSE scripts can make use of the service detection results, allowing them to produce more accurate or targeted output.

NSE (Nmap Scripting Engine)

Study Notes

What is NSE?

- NSE stands for **Nmap Scripting Engine**.
- It allows Nmap to perform tasks beyond simple port scanning by running small programs called **NSE scripts**.
- NSE scripts are written in **Lua**.
- Instead of only identifying open ports, NSE can interact with services and gather additional information or perform specific checks.

Examples:

- Check anonymous FTP login.
- Read HTTP page titles.
- Enumerate SMB shares.
- Retrieve SSL certificate information.
- Gather DNS information.
- Check for certain known vulnerabilities.

High-Level Working of an NSE Script

```
Nmap scans the target
        ↓
Open service is discovered
        ↓
Selected NSE script runs
        ↓
Script sends protocol-specific requests
        ↓
Service replies
        ↓
Script analyzes the response
        ↓
Script returns results to Nmap
        ↓
Nmap displays the output
```

An NSE script is simply a program that communicates with a service and analyzes its responses.

Running NSE Scripts

Run one script:

```
nmap --script ftp-anon <target>
```

Run multiple scripts:

```
nmap --script ftp-anon,http-title <target>
```

Run an entire category:

```
nmap --script vuln <target>
```

Script vs Category

A **script** is one individual `.nse` file.

```
ftp-anon.nse
```

A **category** is a collection of related scripts.

```
vuln
    ├── ftp-vsftpd-backdoor
    ├── http-shellshock
    ├── smb-vuln-ms17-010
    └── many more
```

Running:

```
nmap --script vuln
```

does **not** run every NSE script. It only runs scripts that belong to the **vuln** category and are applicable to the discovered services.

Where NSE Scripts Are Stored

On Kali Linux:

```
ls /usr/share/nmap/scripts
```

Every `.nse` file is one Lua program.

Examples:

```
ftp-anon.nse
http-title.nse
ssh-hostkey.nse
smb-os-discovery.nse
```

Reading an NSE Script

Opening a script is only to understand that:

- It is real Lua code.
- It imports libraries.
- It contains logic.
- It sends requests.
- It processes responses.
- It returns results back to Nmap.

At my current stage, I do **not** need to understand Lua syntax or memorize the code.

Important Parts of an NSE Script

Typical structure:

```
Import libraries
        ↓
Description
        ↓
Categories
        ↓
portrule
        ↓
action()
        ↓
Return results
```

The important concepts are:

- **Categories** determine which category the script belongs to.
- **portrule** determines on which services the script should run.
- **action()** contains the actual logic of the script.
- The script communicates with the target service and returns the results to Nmap.

What 'portrule' Does

Example:

```
portrule = shortport.http
```

Meaning: Only run this script against HTTP services.

```
22 SSH   → Skip
80 HTTP  → Run script
445 SMB  → Skip
```

This explains why service-specific scripts do not run on unrelated services.

ftp-anon Scan

Command:

```
sudo nmap --script ftp-anon <target>
```

My Ubuntu VM did not have an FTP service running (port 21 was closed).

Therefore:

- The script was invoked.
- It found no FTP service.
- It had nothing to test.
- It exited quietly.
- No NSE output was produced.

> **Note:** This is normal behavior. No output does not mean the script failed.

vuln Category Scan

Command:

```
sudo nmap --script vuln <target>
```

The `vuln` category automatically ran multiple applicable vulnerability scripts.

The scan showed HTTP scripts checked for:

- CSRF
- DOM XSS
- Stored XSS

None of these were detected.

> *This does not prove the website is secure. It only means those particular scripts did not detect those particular vulnerabilities.*

The scan also found:

```
/info.php
```

This is simply an interesting discovery. As an ethical hacker, this becomes something worth manually investigating. It is **not automatically a vulnerability**.

The SMB scripts produced messages such as:

```
Could not negotiate a connection
```

This means:

- The script could not complete its test.
- It does **not** mean vulnerable.
- It does **not** mean safe.

> **Note:** No conclusion should be drawn from that result alone.

Pre-scan Script Result

The scan showed:

```
224.0.0.251
```

This is **not another host**. It is the IPv4 multicast address used for **mDNS/Avahi**.

The `broadcast-avahi-dos` script communicated using this multicast address to discover hosts that participate in the mDNS service.

So `224.0.0.251` is a multicast destination, **not** an individual machine's IP address.

Ethical Hacker's Interpretation of the vuln Scan

The correct conclusion is:

- The target exposes SSH, HTTP and SMB services.
- The automated NSE vulnerability scripts did not identify any confirmed vulnerabilities.
- `/info.php` was discovered and may be worth manual inspection.
- Some SMB vulnerability checks were inconclusive because the scripts could not complete their tests.

> *I should never conclude: "The target is secure."*

NSE provides an initial assessment, not a complete security audit.

Why Inspect /info.php?

After discovering:

```
/info.php
```

I would manually visit:

```
http://<target>/info.php
```

or use:

```
curl http://<target>/info.php
```

The purpose is to inspect what the server returns.

The NSE script only discovered that the resource exists—it did not analyze its full contents.

curl

`curl` is an HTTP client. It:

- establishes a TCP connection,
- sends an HTTP request,
- receives the HTTP response,
- parses the HTTP response,
- prints the response body.

Flow:

```
curl
   ↓
TCP connection
   ↓
HTTP GET request
   ↓
```

```
Server executes PHP (if needed)
   ↓
Server sends HTTP response
   ↓
curl receives it
   ↓
Response body printed to terminal
```

No browser is involved.

Browser vs curl

Browser	curl
Server → Browser	Server → curl
Parse HTML	Print response body
Execute JavaScript	—
Apply CSS	—
Render webpage	—

`curl` does **not** render HTML. It simply displays what the server returned.

PHP and curl

PHP executes **only on the server**.

Example:

```
info.php
   ↓
PHP interpreter executes it
   ↓
Generated HTML
   ↓
HTML sent to client
```

The client never receives the PHP source code (assuming correct server configuration).

`curl` receives the generated output (such as HTML), not the PHP code itself.

Components Involved When Using curl

```
curl
   ↓
Kali TCP/IP stack
   ↓
Network
   ↓
Ubuntu TCP/IP stack
   ↓
Apache/Nginx
   ↓
PHP (if required)
   ↓
Apache/Nginx
   ↓
Ubuntu TCP/IP stack
   ↓
Network
   ↓
Kali TCP/IP stack
   ↓
curl
   ↓
```

```
Terminal
```

A browser is not involved anywhere in this process.

What I Need to Know About NSE at My Current Stage

I should know:

- What NSE is.
- What an NSE script is.
- Difference between a script and a category.
- How to run scripts.
- How to run categories.
- Why `-sV` is commonly paired with NSE.
- How to interpret NSE output.
- That NSE scripts are Lua programs.
- That scripts contain logic for communicating with services.

I do not need to:

- Memorize Lua syntax.
- Memorize script source code.
- Learn to write NSE scripts yet.

Reading the source code was useful only to understand that NSE scripts are ordinary programs that send requests, process responses, and return results to Nmap.